

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Sunday 13th September, 2026

Chapter 1

Clustering Agent Representations

As discussed in the previous chapter, current explainability methods for Reinforcement Learning (RL) still lack intrinsic approaches capable of explaining the internal mechanisms of deep neural policies. To the best of our knowledge, only [Zahavy et al., 2017] have proposed exploiting the Latent Space (LS) learned by neural policies in order to identify the concepts used by the agent. Its main idea is to analyze the Latent Representation (LR) produced by the Neural Network (NN) and organize them into clusters that correspond to high-level concepts.

While the core idea of this paper is closely aligned with our vision of explainability in RL, this approach suffers from two important limitations. First, the clusters are manually constructed. Such a procedure heavily depends on the user’s expertise and neither guarantees reproducibility or scales. Second, no evaluation protocol is proposed to assess the quality of the obtained clusters. Their relevance is essentially determined through visual inspection.

The Clustering Agent Representation (CAR) method proposed in this chapter addresses these two limitations. First, it automates concept discovery by relying on unsupervised clustering algorithms to identify the Concept Structure (CS) of the LS. Second, it introduces an objective evaluation metric based on the stability of the extracted representations across independently trained policies.

This chapter is organized as follows:

- Section 1.1 lays out the theoretical assumptions underlying the proposed CAR method, which we term the *Representation Convergence Hypothesis* (Hyp. 1).
- Section 1.2 then details the CAR method itself, covering both the extraction of LR using surrogate policies and the automatic detection of CS through clustering.
- Section 1.3 introduces the metrics we propose for evaluation.
- Section 1.4 situates CAR within the broader landscape of mechanistic interpretability.

1.1 Representation Convergence Hypothesis

The CAR method relies on a assumption, referred to as the *Representation Convergence Hypothesis* (Hyp. 1). This hypothesis follows the two assumptions established previously throughout this manuscript.

The *Mechanistic Hypothesis* (Hyp. ??) rejects the notion that deep NN constitute irreducible black boxes. Adopting a mechanistic perspective, we consider that every process results from a sequence of causal computations. Although these computations may be complex, they remain explainable through the analysis of the internal mechanisms.

The *Concept Hypothesis* (Hyp. ??) states that NN achieve generalization by learning concepts. During optimization, the network progressively organizes its LS into representations that capture the information required to solve the task.

Following these two hypotheses, it becomes possible to explain a NN by studying the organization of its LS. This family of approaches is referred to as mechanistic interpretability. Since the CAR method is grounded in these two hypotheses, it falls within this field of research. More specifically, CAR focuses on a specific aspect of mechanistic interpretability known as universality. Universality refers to the consistency with which a concept is recovered across different models trained on the same task [Templeton et al., 2024].

To provide a explanation for this phenomenon, we developed the *Representation Convergence Hypothesis* (Hyp. 1). This hypothesis states that, when NN are trained to solve the same task, a set of concepts should systematically emerge because these concepts provide a effective way of representing the problem. As a result, independently trained models should develop similar concepts in their learned representations.

Hypothesis 1: Representation Convergence Hypothesis

For a given task, independently trained NN tend to develop similar sets of concepts because these concepts provide an efficient representation for solving the task.

The CAR method is designed to investigate the *Representation Convergence Hypothesis* (Hyp. 1) by assessing whether it can consistently recover similar concepts across independently trained models. The hypothesis therefore serves both as the foundation of the method and as the phenomenon it aims to investigate. If CAR consistently recovers the same set of concepts across different training runs, this set is more likely to reflect invariant properties of the task than model-specific artifacts. Conversely, the absence of representation convergence would provide evidence against the hypothesis and would also question the validity of the CAR approach.

It is important to clarify that this hypothesis does not imply convergence at the parameter level. Independently trained models are expected to converge toward different sets of weights. The proposed hypothesis concerns a abstract level of representation. We assume that the conceptual organization induced by their LR converges toward an equivalent CS, even though the numerical implementation of the networks differ.

The remainder of this chapter develops the methodology used to investigate this hypothesis.

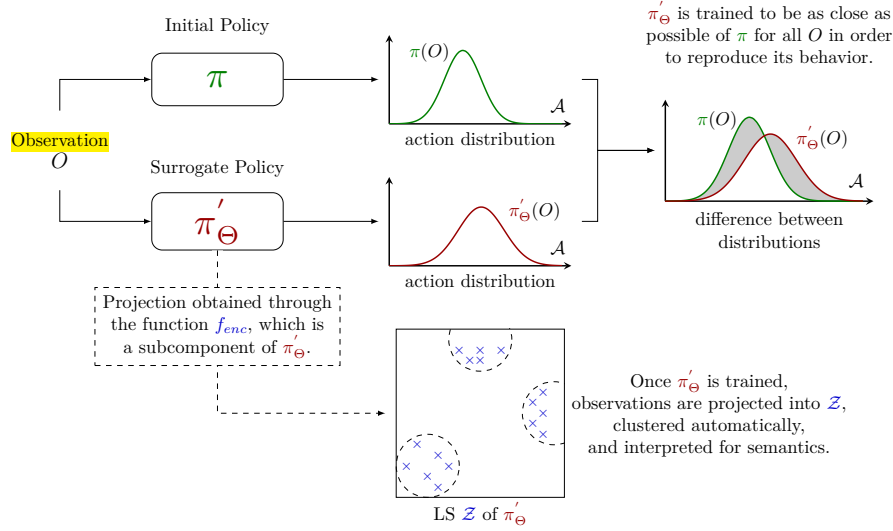


Figure 1.1: Illustration of the Clustering Agent Representation method.

1.2 Latent Clustering of Surrogate Policies

Following the work of [Zahavy et al., 2017], we analyze the LS through clustering techniques, placing our approach within the broader field of Deep Clustering. This design choice is motivated by our rejection of the *Superposition Hypothesis* (Hyp. ??) and, more generally, of any view that assumes a linearly organized CS, as discussed in Section ??.

The CAR methodology is composed of two main stages. The first stage trains multiple independent NN, called surrogate policies, which implement the same decision policy. The second stage clusters their LR in order to identify the underlying conceptual organization. Once these two stages have been completed, the resulting clustering can be analyzed from two complementary perspectives. First, the consistency of the resulting clusters across independent training runs can be assessed to determine whether the identified clusters are stable. Second, the semantic content of the clusters can be examined to determine whether they correspond to meaningful concepts. Figure 1.1 provides a visual overview of the two stages, from the imitation of the initial policy to the clustering of the surrogate representations.

This section is organized into three parts. The first explains why multiple surrogate policies are used to study the stability of the representations. The second describes how these policies are trained to match the initial policy as closely as possible. The third presents how their LR are clustered to identify the CS.

1.2.1 Multiple surrogate policies

The CAR method was developed to test the *Representation Convergence Hypothesis* (Hyp. 1). This requires training several NN independently and comparing their LR. Consistent similarities in the resulting CS would provide empirical support for this hypothesis.

In supervised learning, this protocol is straightforward since several models can be independently trained on the same target function. RL introduces an additional difficulty. Because of stochastic exploration, independently trained agents are not guaranteed to converge toward the same policy. Even if they achieve comparable performance, they may solve the task using different strategies. This makes it difficult to determine whether differences in the LS result from different representations of the same strategy or from differences in the learned strategies themselves.

To overcome this limitation, we consider an already trained initial policy. This policy is then used as a target for imitation learning [Ross et al., 2011]. In this process, several surrogate NN are independently trained in a supervised manner to reproduce its behavior. This makes it possible to investigate whether equivalent policies also exhibit equivalent conceptual organizations.

It is important to emphasize that the use of surrogate policies does not make CAR a surrogate explainability method, as defined in Subsection ???. The surrogate policies remain deep NN. Their role is not to simplify the decision process but to provide multiple independent realizations of the same policy function, enabling the study of representation convergence.

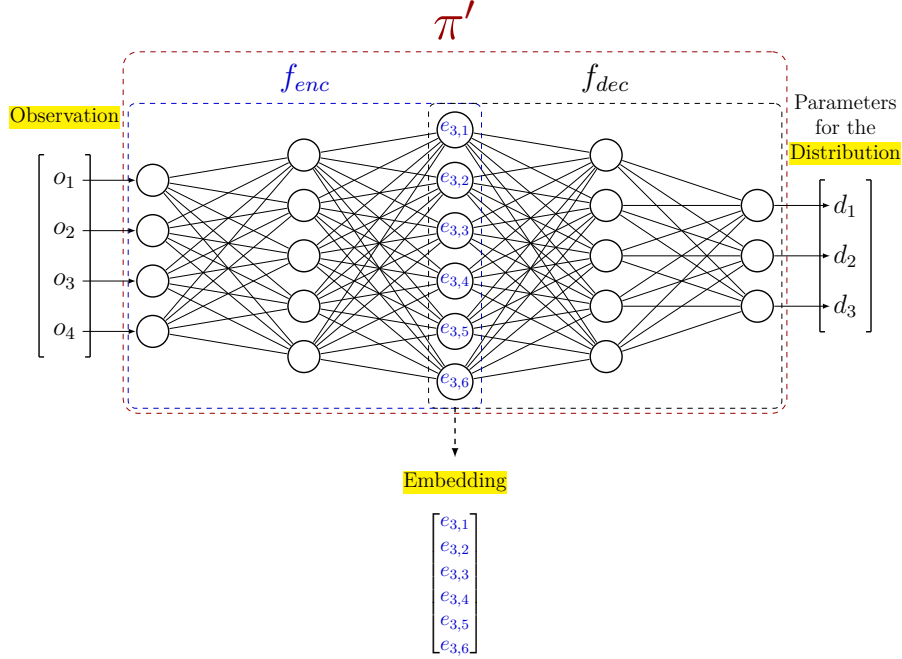
1.2.2 Training surrogate policies

Throughout the remainder of this manuscript, the initial policy we aim to explain is denoted as π . This initial policy is obtained without imposing any constraints on its origin. It may result from classical or deep reinforcement learning [Sutton and Barto, 2018], alpha-beta algorithm [Knuth and Moore, 1975], or even from demonstrations provided by humans [Ross and Bagnell, 2010]. The crucial requirement is the availability of a dataset containing a sufficient number of projections from the observation space $\mathcal{O}$ to the action space $\mathcal{A}$ induced by π to train a NN.

The replicated policies are referred to as surrogate policies. To assess the stability of the learned representations, n surrogate policies are trained independently, denoted by $\pi'_1, \dots, \pi'_n$. All surrogate policies share the same architecture. It consists of a NN with two components, an encoder f_{enc} and a decoder f_{dec} , highlighted by the blue and black dashed rectangles, respectively, in Figure 1.2. The encoder maps the observation O to the embedding, and the decoder reconstructs the parameters of the action distribution from this embedding. For a given surrogate policy π'_i , parameterized by Θ_i , its encoder and decoder are written f_{enc_i} and f_{dec_i} .

To train each surrogate policy π'_i to accurately replicate the action distribution, we define a loss function $\mathcal{L}$ optimized via gradient descent as introduced in Section ???. This loss, defined in Equation (1.1), is computed as the Mean Squared Error (MSE) between the parameters of the action distributions, obtained by applying the function p , which maps a distribution to its parameterization. In practice, this function p does not need to be explicitly defined, as neural policies directly output the parameters of the action distribution.

$$\min_{\Theta_i} \sum_{O \in \mathcal{O}} \text{MSE}\left(p(\pi(O)), p(\pi'_{\Theta_i}(O))\right) \quad (1.1)$$

Figure 1.2: Example of a surrogate policy architecture π' .

1.2.3 Cluster extraction

Once the surrogate policies are trained, their encoders are used to generate a point cloud from a set of observations. This point cloud provides a sample of the LS to be analyzed. The proposed CAR method adopts the deep clustering approach introduced in Section ???. Under this approach, conceptually similar observations are expected to produce nearby LR. A CS can therefore be viewed as a region of the LS containing related LR. To identify these regions, a clustering algorithm is applied to the point cloud.

The clustering is performed independently for each surrogate policy. For a surrogate policy π'_i , applying its encoder f_{enc_i} to a subset of observations $\mathbb{O} \subseteq \mathcal{O}$ yields the set of embeddings $\mathcal{E}_i$. A clustering function f_{clust} then partitions the point cloud $\mathcal{E}_i$ into groups of nearby LR, assigning each observation a cluster label and yielding the partition $\mathcal{C}_i$. The same observation set $\mathbb{O}$ is used for every surrogate policy, so that the partitions $\mathcal{C}_1, \dots, \mathcal{C}_n$ are defined over the same observations and can be compared. This comparison is the basis of the evaluation introduced in the following section. Algorithm 1 states the complete CAR procedure.

Two choices govern this analysis without being part of the clustering algorithm itself. The first is the choice of layer used to extract the representations. Each hidden layer defines its own LS, reflecting the information represented at that stage of the network. CAR can therefore be applied to the LR extracted from any layer. This allows the conceptual organization of the policy to be examined at different levels of the network.

The second is the number of clusters. CAR does not assume that a task

Algorithm 1: Clustering Agent Representation (CAR)**Input:**Initial policy π Observation dataset $\mathbb{O}$ Number of surrogate policies n Surrogate policy architecture π' (encoder f_{enc} , decoder f_{dec})Learning rate η Clustering function f_{clust} **Output:**Trained surrogate policies $\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}$ Clusterings $\mathcal{C}_1, \dots, \mathcal{C}_n$ *Behavioral cloning dataset built from the target policy π* **1** $\mathcal{D} \leftarrow \{(O, p(\pi(O))) \mid O \in \mathbb{O}\};$ *MSE loss between action-distribution parameters, as defined in Equation (??)***2** $\mathcal{L} \leftarrow \text{MSE};$ *Independently train n surrogate policies by behavioral cloning***3** **for** $i = 1, \dots, n$ **do****4** Initialize Θ_i randomly;*Behavioral cloning of π via SGD, see Algorithm ??***5** $\Theta_i \leftarrow \text{SGD}(\mathcal{D}, \pi', \mathcal{L}, \eta, \Theta_i);$ *Cluster the latent representations of each surrogate policy***6** **for** $i = 1, \dots, n$ **do***Embeddings produced by the encoder of π'_{Θ_i}* **7** $\mathcal{E}_i \leftarrow \{f_{enc_i}(O) \mid O \in \mathbb{O}\};$ *Partition of the embedded observations into clusters***8** $\mathcal{C}_i \leftarrow f_{clust}(\mathcal{E}_i);$ **9** **Return** $(\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}), (\mathcal{C}_1, \dots, \mathcal{C}_n)$

172 admits a single correct number of concepts. This choice instead determines the
 173 granularity at which the conceptual organization is examined. A small number
 174 reveals broad, strategic concepts, while a larger number reveals finer, tactical
 175 ones. The influence of both choices is studied in Chapter 2.

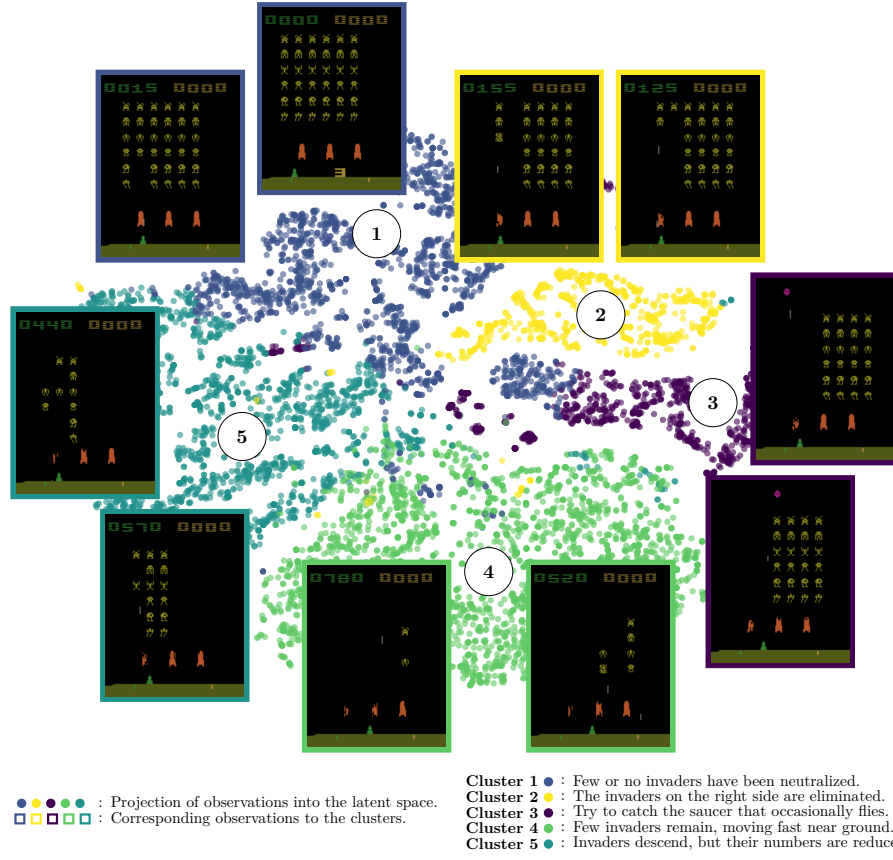


Figure 1.3: Two-dimensional projection of the LS embeddings of a surrogate policy trained on Space Invaders, together with the CAR clusters and their corresponding observations.

Figure 1.3 shows the outcome of this procedure for a surrogate policy trained on Space Invaders, using five clusters. The clusters are computed in the original high-dimensional LS. The point cloud is a two-dimensional projection used only for visualization. Each cluster gathers observations that correspond to a distinct, interpretable stage of a game:

- **Cluster 1:** The start of a game, with few or no invaders neutralized.
- **Cluster 2:** The right-hand group of invaders eliminated.
- **Cluster 3:** The pursuit of the bonus saucer.
- **Cluster 4:** The end of a game, with a few fast invaders close to the ground.
- **Cluster 5:** The middle of a game, with the invaders descending but already thinned out.

The position of an observation in the LS thus reflects the stage of the environment it belongs to, suggesting a correspondence between the geometry of the LS and the semantic properties of environment states.

This example illustrates what CAR is expected to produce, but it relies on the visual inspection of a single surrogate policy, which is precisely the type of evidence whose limitations motivated this work. Three aspects must therefore be evaluated before such an analysis can be trusted. First, the surrogate policy must faithfully reproduce the initial policy so that its explanations remain relevant to the target policy. Second, the recovered conceptual organization must be consistent across independently trained surrogate policies. Finally, the clusters must capture genuine abstractions rather than simply reproducing structure already present in the observations or action distributions.

The following section introduces one evaluation metric for each of these aspects and details how it is computed.

1.3 Evaluation Metrics

To evaluate the effectiveness of CAR, four evaluation metrics are introduced:

- ***Behavior Reconstruction*** : it measures how well a surrogate policy faithfully reproduces the behavior of the initial policy by comparing their cumulative rewards.
- ***Clustering Universality*** : it evaluates the stability of clusterings by measuring the similarity of clusters obtained from independently trained surrogate policies.
- ***Abstraction Score*** : it quantifies the extent to which LS clusters capture abstractions rather than directly reflecting the structure of observations, actions, or the network architecture prior to training.
- ***Adjusted Clustering Universality*** : it combines the two previous metrics into a single measure of how much of the cross-seed agreement captured by *Clustering Universality* cannot be explained away by any of the trivial baselines underlying *Abstraction Score*.

The remainder of this section details the motivation for each metric and describes how they are calculated. In the first subsection, we develop the *Behavior Reconstruction*. We then introduce the Adjusted Rand Index (ARI) measure, which serves as the foundation for the three other metrics introduced in the subsequent subsections, namely *Clustering Universality*, *Abstraction Score*, and *Adjusted Clustering Universality*.

1.3.1 Behavior Reconstruction

We introduce the *Behavior Reconstruction* metric to assess whether π' faithfully reproduces the behavior of π . This metric compares the returns obtained by the initial policy π and the surrogate policies π' over a set of evaluation episodes. We use the definition of return introduced in Section ??, where $g(t, \gamma)$ denotes the discounted sum of rewards collected along a trajectory t . Here, the return is computed over complete episodes with $\gamma = 1$, so g corresponds to the total reward collected during the episode. Each policy is evaluated over m episodes. We denote by g_{π}^i the return obtained by the initial policy in episode i , and by $g_{\pi'_j}^i$ the return obtained by surrogate policy π'_j in episode i . The *Behavior*

233 *Reconstruction* is then defined as the mean return of the surrogate policies
 234 normalized by the mean return of the initial policy:

$$\text{Behavior Reconstruction} = \frac{1}{n \sum_{i=1}^m g_{\pi}^i} \sum_{j=1}^n \sum_{k=1}^m g_{\pi'_j}^k \quad (1.2)$$

235 In imitation learning, it is unlikely to expect the surrogate policy π' to
 236 outperform the initial policy π it aims to approximate through supervised
 237 learning [Ross and Bagnell, 2010]. Consequently this metric ranges in $[0, 1]$.
 238 Values approaching 1 indicate minimal loss in reproducing π , while values close
 239 to 0 indicate poor reproduction.

240 We deliberately adopt a reward-based definition rather than alternatives based
 241 on actions or action distributions (e.g., training loss, accuracy, KL divergence
 242 [Shlens, 2014], or optimal transport [Peyré and Cuturi, 2020]). Since surrogate
 243 policies are trained in a supervised manner, they cannot develop strategies
 244 beyond those demonstrated by the initial policy. This means that if the rewards
 245 achieved are comparable, the surrogate policy is effectively copying the initial
 246 policy's behavior.

247 1.3.2 Adjusted Rand Index

248 Within CAR, concepts are assumed to correspond to clusters in the LS. Combined
 249 with the *Representation Convergence Hypothesis* (Hyp. 1), this leads to a direct
 250 experimental prediction. If independently trained surrogate policies converge
 251 toward the same conceptual organization, clustering their LR should produce
 252 similar partitions.

253 When a particular observation is processed by the network, it is expected to
 254 activate the same concept regardless of the specific neural implementation of the
 255 policy. If independently trained policies converge toward the same conceptual
 256 organization, a given observation should activate the same concept across all
 257 models. As a consequence, observations grouped together within a cluster for
 258 one policy should also be grouped together for another independently trained
 259 policy.

260 In the clustering literature, the most used measures for comparing two
 261 partitions is the ARI [Hubert and Arabie, 1985]. This measure quantifies the
 262 agreement between cluster assignments independently of the spatial organiza-
 263 tion of the clusters. Both metrics introduced later in this chapter, *Clustering*
 264 *Universality* and *Abstraction Score*, rely on the ARI. Since the ARI is a complex
 265 measure, we review its definition and discuss its main properties.

The ARI is derived from the Rand Index (RI) [Rand, 1971], which measures the agreement between two partitions by considering every possible pair of points in the point cloud. For each pair, four situations are possible:

- **Agreement:** The points belong to the same cluster in both partitions.
- **Agreement:** The points belong to different clusters in both partitions.
- **Disagreement:** The points belong to the same cluster in the first partition but to different clusters in the second.
- **Disagreement:** The points belong to different clusters in the first partition but to the same cluster in the second.

The RI measures the proportion of pairs for which the two partitions agree:

$$\text{RI} = \frac{\text{number of agreeing pairs}}{\text{total number of pairs}}.$$

Although the RI is intuitive, it suffers from an important limitation. In most clustering problems, the majority of pairs of points belong to different clusters. Consequently, even two completely independent partitions will agree on a large number of pairs, simply because both assign them to different clusters. This effect becomes more pronounced as the number of points or clusters increases.

The ARI addresses this issue by accounting for the agreement that would be expected purely by chance. It compares the observed agreement RI to the expected agreement under random partitions $\mathbb{E}[\text{RI}]$. The score is then normalized by the maximum agreement achievable beyond this random baseline, $\max(\text{RI}) - \mathbb{E}[\text{RI}]$. As a result, an ARI of 0 corresponds to the level of agreement expected from random clusterings, while an ARI of 1 indicates perfect agreement between the two partitions:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}$$

In practice, the ARI is computed directly from the cluster assignments of the two partitions. Let clustering A be composed of partitions $\{A_i\}$ and clustering B of partitions $\{B_j\}$. Let $a_i = |A_i|$ denote the number of samples in partition i of clustering A , and $b_j = |B_j|$ the number of samples in partition j of clustering B . Furthermore, let $n_{ij} = |A_i \cap B_j|$ be the number of samples shared by partition i of clustering A and partition j of clustering B , and let n be the total number of samples. With these definitions, the ARI can be computed as:

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}$$

Because it is bounded by 1, a value such as 0.5 can be misleadingly interpreted as “halfway to a perfect match.” This interpretation is incorrect because the ARI is normalized against the agreement expected from two random partitions. As a result, values close to 1 require the two clusterings to be nearly identical, while a value of 0.5 already indicates substantial agreement [Amezquita et al., 2020].

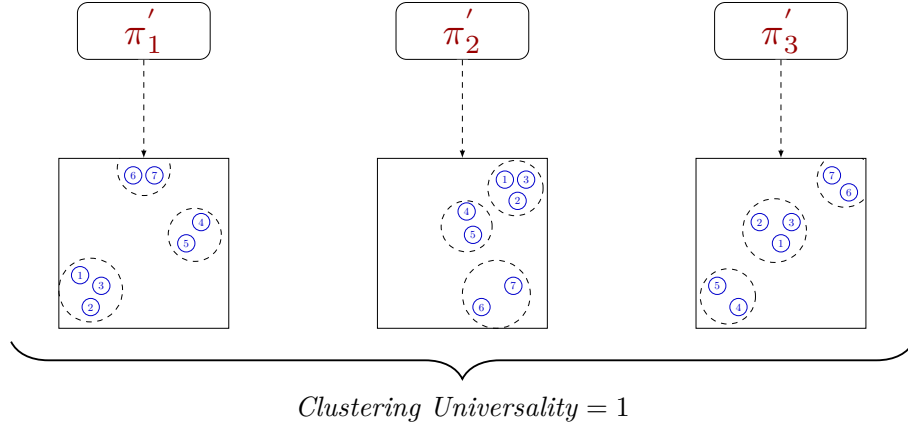


Figure 1.4: Illustration of *Clustering Universality* in the ideal case where projections of the same observations are grouped within the same clusters across all surrogate policies.

1.3.3 Clustering Universality

Having introduced the ARI, we can now define the *Clustering Universality* metric. While the ARI quantifies the similarity between two partitions, the objective of CAR is to assess the consistency of conceptual organizations across an entire set of independently trained surrogate policies.

To this end, the same set of observations is projected into the LS of each surrogate policy and clustered. Under the *Representation Convergence Hypothesis* (Hyp. 1), these independently obtained partitions are expected to be similar. *Clustering Universality* quantifies this by computing the mean pairwise ARI:

$$\text{Clustering Universality} = \frac{1}{\binom{n}{2}} \sum_{i=2}^n \sum_{j=1}^{i-1} \text{ARI}(\mathcal{C}_i, \mathcal{C}_j). \quad (1.3)$$

This metric takes values in $[-1, 1]$. A value close to 1 indicates that surrogate policies produce highly similar partitions, suggesting that the clusters reflect intrinsic properties of the policy. Conversely, a value near 0 implies that the similarity between partitions is no better than that of two randomly generated clusterings, meaning that the clustering fails to capture any meaningful structure in the policy. A negative value, on the other hand, signals that the agreement between partitions is worse than what would be expected by chance, indicating a systematic disagreement between the clusterings.

Figure 1.4 illustrates *Clustering Universality* in the ideal case. It shows three LS obtained by projecting the same seven observations, labeled 1 to 7, through three independently trained surrogate policies. Although the embeddings occupy different positions, the clusters group exactly the same elements in all three cases. The partitioning is therefore identical, resulting in an ARI of 1.

1.3.4 Abstraction Score

If *Clustering Universality* is high, it becomes important to determine what underlies this stability. Two possibilities arise:

- (i) the LS offers no abstraction, as defined in Definition ??, either because it merely reproduces the structure of the observation space or action distribution, or because it reproduces a structure that is already present in the network architecture prior to any training;
- (ii) the LS captures similarities in decision-making processes, yielding behaviorally meaningful clusters.

To distinguish among these cases, we introduce the *Abstraction Score*. This metric contextualizes latent clusterings by examining how their patterns align with three reference baselines: the raw observation data, the action data, and an untrained network of the same architecture. We consider : $\mathcal{C}_{obs}$, the clustering in the observation space $\mathbb{O}$, and $\mathcal{C}_{act}$, the clustering in the action-space distribution parameters. The clusterings $\mathcal{C}_{obs}$ and $\mathcal{C}_{act}$ are performed using the same algorithm, distance metric, and hyperparameters as those used for $\mathcal{C}_i$ in the LS, ensuring that any biases introduced by the clustering procedure itself are minimized.

The third baseline addresses the case where the LS merely reflects an inductive bias of the network architecture rather than something learned. To this end, we introduce m untrained networks sharing the same architecture as the surrogate policies, but with randomly initialized, never-updated weights. Each of these m untrained networks is used to project the same observations into its own latent space, which is then clustered in the same way, yielding partitions $\mathcal{C}_{untr,1}, \dots, \mathcal{C}_{untr,m}$. This baseline isolates what the architecture already imposes on the representation before any learning takes place, so that a high similarity between $\mathcal{C}_i$ and this baseline indicates that the corresponding structure cannot be attributed to training.

For each latent clustering $\mathcal{C}_i$, we define its similarity to these three baselines using the ARI:

$$\begin{aligned} A_i^{obs} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{obs}) \\ A_i^{act} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{act}) \\ A_i^{untr} &= \frac{1}{m} \sum_{k=1}^m \text{ARI}(\mathcal{C}_i, \mathcal{C}_{untr,k}) \end{aligned}$$

The *Abstraction Score* is subsequently calculated as one minus the average of the maximum absolute similarity values across these three reference baselines:

$$\text{Abstraction Score} = 1 - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{obs}|, |A_i^{act}|, |A_i^{untr}|) \quad (1.4)$$

The *Abstraction Score* ranges from $[0, 1]$. A score around 0 means that the latent clusters are strongly aligned with the observation space, the action space, or the untrained network, reflecting no abstraction, case (i). A score close to 1 indicates that the latent clusters are independent of the observation space, the action space, and the network’s initialization, capturing abstractions that meaningfully reflect the agent’s behavior, case (ii).

1.3.5 Adjusted Clustering Universality

As discussed above, *Clustering Universality* alone does not indicate whether the recovered structure is meaningful. We therefore introduce the *Adjusted Clustering Universality*, which measures how much of the agreement between the different surrogate policies remains after accounting for the three trivial baselines.

$$\text{Adjusted Clustering Universality} = \text{Clustering Universality} - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{\text{obs}}|, |A_i^{\text{act}}|, |A_i^{\text{untr}}|) \quad (1.5)$$

This metric ranges in $[-1, 1]$. A value close to 1 indicates that surrogate policies converge toward a partition that is both highly reproducible across seeds and clearly not explained by the observation space, the action space, or the network’s initialization, providing the strongest evidence for *Representation Convergence Hypothesis* (Hyp. 1). A value close to 0, or negative, indicates that whatever agreement exists between independently trained surrogate policies is largely trivial: the surrogate policies converge, but toward a partition that could just as easily emerge from raw pixel similarity, from the action taken, or from an untrained network sharing the same architecture.

1.4 Conclusion

The CAR method implements the three hypotheses developed in this manuscript, the *Mechanistic Hypothesis* (Hyp. ??), the *Concept Hypothesis* (Hyp. ??), and the *Representation Convergence Hypothesis* (Hyp. 1). It assumes that neural policies can be explained through their internal mechanisms, that these mechanisms are organized as concepts in the LS, and that independently trained policies converge toward an equivalent conceptual organization. The *Clustering Universality* and *Abstraction Score* metrics turn these assumptions into measurable quantities, which makes the hypotheses empirically testable. The experimental study reported in Chapter 2 provides support for this hypothesis-driven approach.

Beyond its role in testing these hypotheses, CAR provides a general and automated method for analysing agent representations. Both the analysis procedure and the metrics are fully automated, requiring no manual intervention or environment-specific adjustments. This makes CAR applicable to a wide range of RL settings.

Chapter 2

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to consistently identify meaningful conceptual structures. The remainder of this chapter is organized as follows:

- Section 2.1 describes the software implementation of the CAR pipeline.
- Section 2.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 2.3 presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

2.1 Software Implementation

For Deep Learning (DL) and Reinforcement Learning (RL), theoretical results are often difficult to apply to real-world cases. The effectiveness of methods in these fields is therefore primarily assessed through experimentation. Consequently, particular attention is given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 2.1. Each stage is an independent executable, and stages communicate through artifacts written to disk. This allows each stage to be rerun or inspected independently. The parameters of each experiment are gathered in a single configuration object, described in Section 2.2. Each result can therefore be traced back to the settings that produced it. The following subsections describe the purpose of each stage, its implementation, and the libraries used.

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

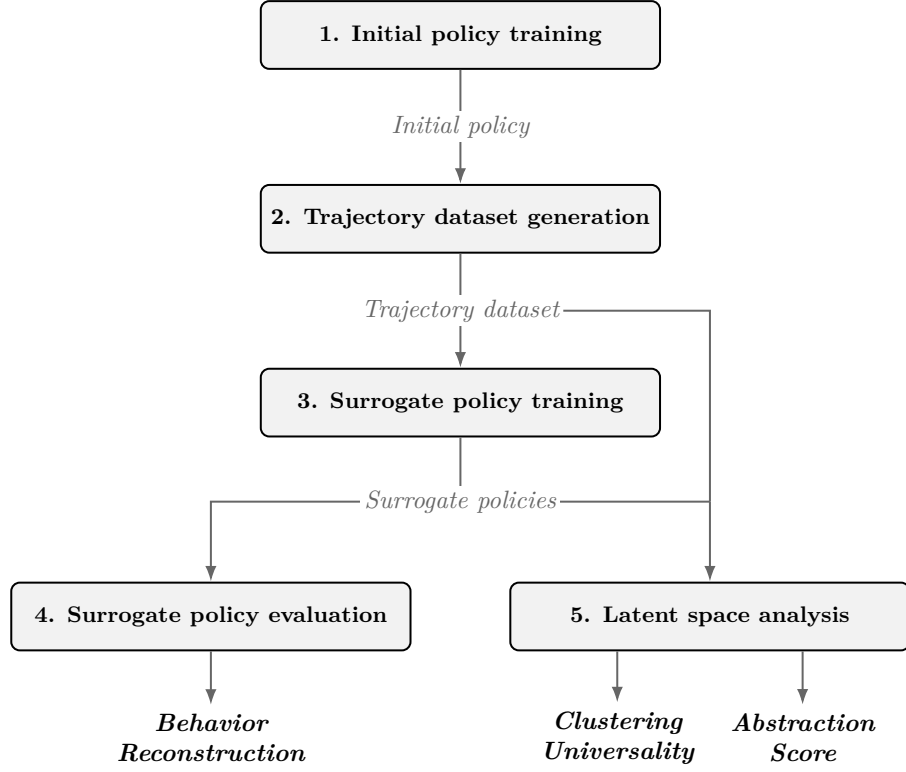


Figure 2.1: The CAR pipeline.

2.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [Towers et al., 2024]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch is the standard deep learning frameworks and is widely used for developing and training neural networks. It provides GPU acceleration and supports a wide range of neural network architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

The resulting policy and critic parameters are saved as a checkpoint together with the environment and training configurations.

433 2.1.2 Trajectory dataset generation

434 The second stage builds the dataset used to train the surrogate policies. RLlib
435 is used to collect trajectories in parallel using multiple environment runners. For
436 each step, the dataset stores the observation together with the parameters of
437 the action distribution produced by the initial policy.

438 Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar
439 structure allows efficient data access and supports concurrent processing. This is
440 useful here because the datasets contain around one million observation–action
441 pairs and cannot be loaded entirely into memory.

442 The resulting dataset is used both to train the surrogate policies and to
443 generate the latent representations analyzed by CAR.

444 2.1.3 Surrogate policy training

445 The third stage trains the surrogate policies using behavior cloning on the
446 trajectory dataset. This provides a supervised learning approach for replicating
447 the initial policy. The training procedure is implemented using PyTorch Lightning
448 [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training
449 neural networks with PyTorch. It handles common training operations such as
450 data loading, optimization, checkpointing, and GPU execution.

451 Each surrogate policy is saved as a checkpoint together with the configuration
452 used for training.

453 2.1.4 Surrogate policy evaluation

454 The fourth stage evaluates how faithfully the surrogate policies reproduce the
455 behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and
456 evaluated in the original environment. Its mean cumulative reward over multiple
457 episodes is compared with that of the initial policy. Evaluation episodes are
458 collected using parallel environment runners.

459 This stage computes the *Behavior Reconstruction*. It therefore verifies that
460 the surrogate policies reproduce the behavior of the initial policy sufficiently well
461 before their latent representations are analyzed. If a surrogate does not meet
462 the required level of behavior reconstruction, its training can be repeated.

463 2.1.5 Latent space analysis

464 The fifth stage performs the main analysis of CAR. The trajectory dataset
465 and trained surrogate policies are used to generate the latent representations.
466 These representations form the point clouds that are analyzed using clustering
467 algorithms.

468 The clustering step relies on the cuML library [RAPIDS, 2023], which pro-
469 vides GPU-accelerated clustering algorithms. This reduces the computational
470 cost and allows point clouds containing tens of thousands of representations to
471 be analyzed.

472 The resulting partitions are used to compute the *Clustering Universality*
473 and the *Abstraction Score*. The stage also produces the material required for
474 the qualitative analysis, including the clustered point clouds and the cluster
475 assignment of each observation.

476 Table 2.1 summarizes the main third-party libraries used in the pipeline.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 2.1: Main software libraries used at each stage of the CAR pipeline.

2.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architecture of the policies, and the main parameters of each pipeline stage.

2.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages. First, Atari is a widely used benchmark in the RL community, with many studies using these environments as standard evaluation settings. This makes the experimental results easier to compare with existing work. Second, these environments were not specifically designed for RL, which reduces the risk of introducing biases related to the design of the environments themselves. Finally, the benchmark provides a diverse set of well-known environments with varying levels of complexity. This diversity makes it possible to evaluate CAR across different types of tasks and levels of difficulty.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017].

In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. An exception is made for Pac-Man, where color information is important for determining the behavior of other agents in the environment. The original color information is therefore preserved for this environment.

To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting

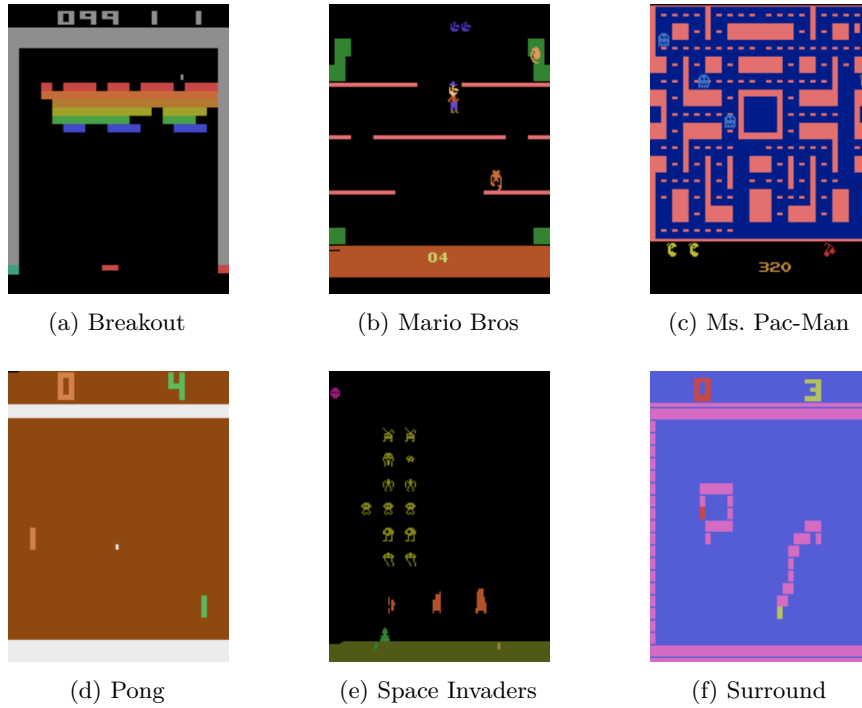


Figure 2.2: The six Atari environments used to evaluate CAR.

observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations rather than relying on a single frame.

Frame skipping is also used to reduce the duration of each episode. The agent selects a new action every five frames, while the selected action is repeated for the intervening frames. This provides sufficient temporal resolution for effective control while reducing the number of decisions required during an episode.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete.

Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

With the environment configuration established, the objective of each environment is described below. Figure ?? provides an overview of the six environments used in the experiments.

Breakout. The agent moves a paddle at the bottom of the screen (Figure 2.2a) to bounce a ball into a wall of bricks, each broken brick scoring a point, and the ball speeding up as the wall is cleared.

Mario Bros. A single-screen platform game (Figure 2.2b). The player clears each phase by hitting enemies from below to flip them, then kicking them off

the platforms before they recover. A central POW block flips every enemy at once but can only be used a few times per phase.

Ms. Pac-Man. The player moves through a maze (Figure 2.2c), eating pellets while avoiding four ghosts, with power pellets briefly making the ghosts edible. The maze structure and the many possible local situations, such as a nearby ghost, an intersection, or an active power pellet, produce a large number of distinguishable states.

Pong. A minimal two-player game (Figure 2.2d) in which the agent controls one paddle and must send the ball past the opponent.

Space Invaders. The agent moves a cannon along the bottom of the screen (Figure 2.2e), firing at descending rows of aliens while dodging their shots and using destructible shields, with a bonus saucer occasionally crossing the top.

Surround. A two-player game on a grid (Figure 2.2f). Each player leaves a solid trail behind a moving cursor, and the goal is to force the opponent to crash into a trail or a wall.

2.2.2 Neural architecture

The surrogate policies use a convolutional neural network to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a multilayer perceptron that produces the parameters of the action distribution.

The architecture of the multilayer perceptron varies across environments. For Pong and Space Invaders, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. Figure 2.3 illustrates both configurations. The size of the output layer matches the number of actions available in each environment: 4 for Breakout, 18 for Mario Bros, 9 for Ms. Pac-Man, 6 for Pong, 6 for Space Invaders, and 5 for Surround.

2.2.3 Parameters

Having defined the environments and neural architectures, the remaining parameters of the CAR pipeline must now be specified. The main parameters shared across the experiments are summarized in Table 2.2. There are too many parameters to discuss them all in detail, as each stage of the pipeline introduces its own configuration choices.

Given the large number of possible configurations, it is not feasible to evaluate all possible combinations. The choices made in this work are therefore based primarily on standard practices from the literature and on practical experience. Whenever possible, we used the simplest suitable configuration. For the clustering algorithm and distance metric in particular, the simplest suitable choices are used to establish a baseline.

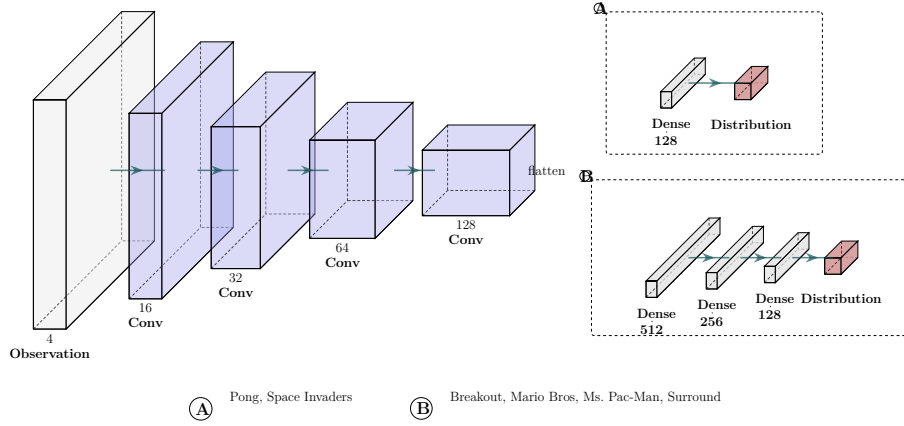


Figure 2.3: Architecture of the surrogate policies.

These choices do not restrict the CAR methodology itself. Most components can be replaced or configured differently depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and determining which configurations produce the most consistent and meaningful results.

2.3 Results

This section reports the results of applying CAR to the six environments. Section 2.3.1 verifies that the initial policies reach a competent level of play, while Section 2.3.2 evaluates whether the surrogate policies faithfully reproduce their behavior using the *Behavior Reconstruction*. Section 2.3.3 analyzes the latent representations using the *Abstraction Score* and *Clustering Universality*.

2.3.1 Initial policy training

Figure 2.4a shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for Pong, to several thousand, for Surround and Ms. Pac-Man. For Ms. Pac-Man the return decreases after its peak. Since the best checkpoint is retained rather than the last one, this does not affect the policy that is subsequently analyzed.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

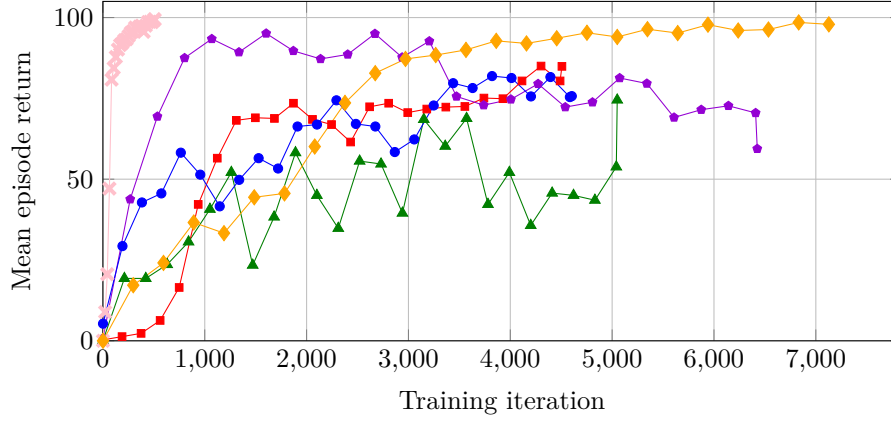
Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	50 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 30

Table 2.2: Main experimental parameters.

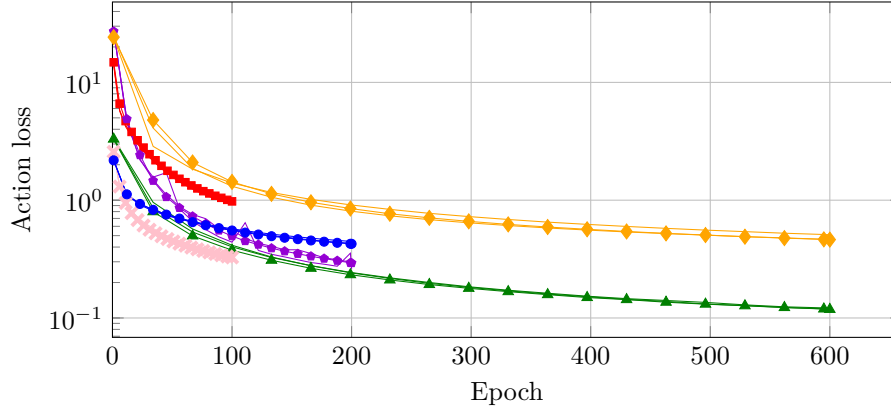
2.3.2 Surrogate policy training

Figure 2.4b shows, for each environment, the training action loss of the three surrogate policies. In every case the loss decreases smoothly, and the three independently trained policies follow almost identical trajectories, converging to the same value.

We further evaluate the ability of the surrogate policies to replicate the behavior of the initial policies using the *Behavior Reconstruction* (Table 2.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed in place of those of the initial policies.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

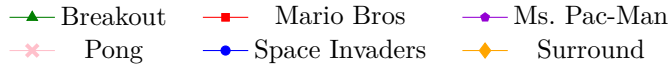


Figure 2.4: Training of the initial and surrogate policies.

2.3.3 Latent space analysis

With the behavioral fidelity of the surrogate policies established, this section analyzes their latent representations using the *Abstraction Score* and *Clustering Universality*, also reported in Table 2.3.

For the remainder of the analysis, a random subset of 50,000 observations is embedded into the Latent Space (LS) of each surrogate policy, using the representation extracted directly after the last convolutional layer, and the resulting point cloud $\mathcal{E}$ is partitioned with k -means clustering [MacQueen, 1967].

Figure 2.5a reports the evolution of *Clustering Universality* with respect to the number of clusters. After the first few values of K , every curve settles onto

Environment	<i>Behavior Reconstruction</i>	<i>Abstraction Score</i>	<i>Clustering Universality</i>
Breakout	0.83 $\pm$ 0.03	0.82 $\pm$ 0.07	0.48 $\pm$ 0.11
Mario Bros	0.99 $\pm$ 0.03	0.71 $\pm$ 0.14	0.55 $\pm$ 0.10
Ms. Pac-Man	1.00 $\pm$ 0.03	0.43 $\pm$ 0.09	0.65 $\pm$ 0.08
Pong	0.95 $\pm$ 0.01	0.99 $\pm$ 0.00	0.49 $\pm$ 0.05
Space Invaders	0.97 $\pm$ 0.07	0.69 $\pm$ 0.06	0.67 $\pm$ 0.07
Surround	1.00 $\pm$ 0.01	0.55 $\pm$ 0.07	0.61 $\pm$ 0.09

Table 2.3: Evaluation of *Behavior Reconstruction*, *Abstraction Score* and *Clustering Universality* across the six Atari environments.

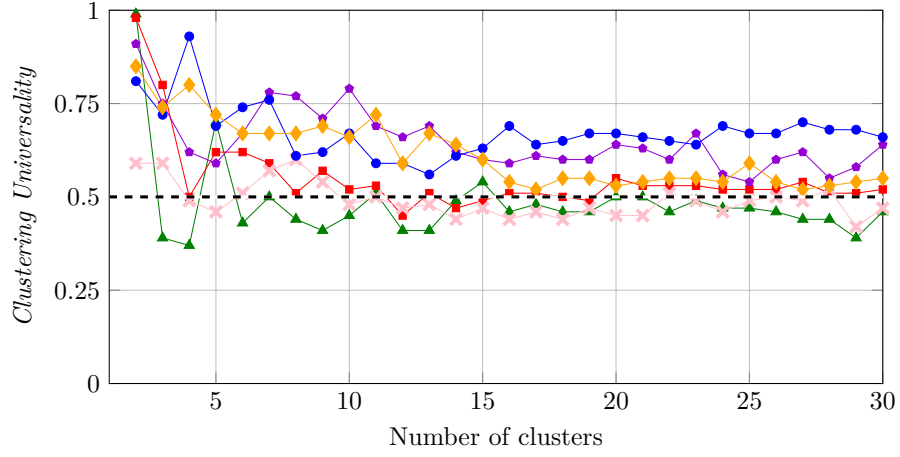
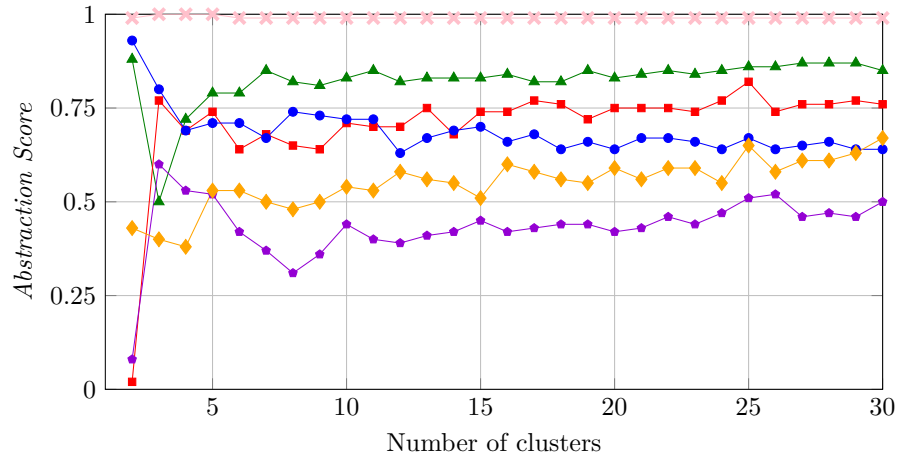
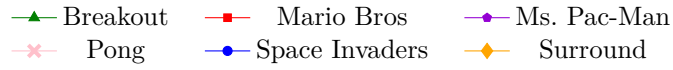
a plateau whose level depends on the environment, from about 0.45–0.50 for Breakout and Pong to about 0.65 for Ms. Pac-Man and Space Invaders. Since *Clustering Universality* is computed from the Adjusted Rand Index (ARI), and an ARI of 0.5 already corresponds to substantial agreement [Amezquita et al., 2020], these values indicate that independently trained surrogate policies recover a similar cluster structure in every environment, and that this structure is stable with respect to the number of clusters. The strength of this agreement varies across environments, but remains clearly above chance everywhere. The absence of a single optimal number of clusters suggests that K should be chosen according to the level of granularity desired for the analysis.

The *Abstraction Score*, in contrast, varies markedly across environments. It is close to 1 for Pong (0.99) and high for Breakout, Mario Bros and Space Invaders (0.69 to 0.82): in these environments the latent clusters are largely independent of a direct clustering of the observations or of the action distributions. It is lower for Surround (0.55) and Ms. Pac-Man (0.43). In these two environments the position of the agent on the screen largely determines both the raw observation and the decision to take, so a clustering of the LS naturally coincides with a clustering of the observations, and the *Abstraction Score* cannot separate the two; a low score does not by itself imply that the representation is not abstract.

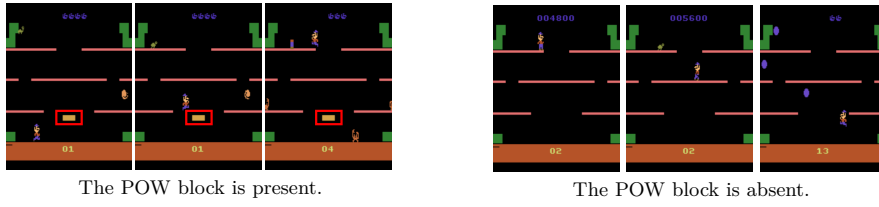
With fewer clusters, CAR highlights broad strategic patterns. For instance, in the Mario Bros environment, the two main groups correspond to whether the POW block is present (highlighted in red in Figure 2.6A) or not. This is particularly interesting, as the use of the POW block is a key determinant of strategy in this environment [AtariAge, 2024].

With a larger number of clusters, the analysis captures more fine-grained tactical behaviors, such as “Attacking the left” and “Navigating through the level” (see Figure 2.6B).

Both perspectives are valid, depending on whether the goal is to study strategic or tactical behaviors. Similar interpretable structures can be observed across the evaluated environments.

(a) Evolution of *Clustering Universality* with respect to the number of clusters.(b) Evolution of *Abstraction Score* with respect to the number of clusters.Figure 2.5: Evolution of *Clustering Universality* and *Abstraction Score* with respect to the number of clusters.

(A) The case with 2 clusters.



(B) The case with 5 clusters.

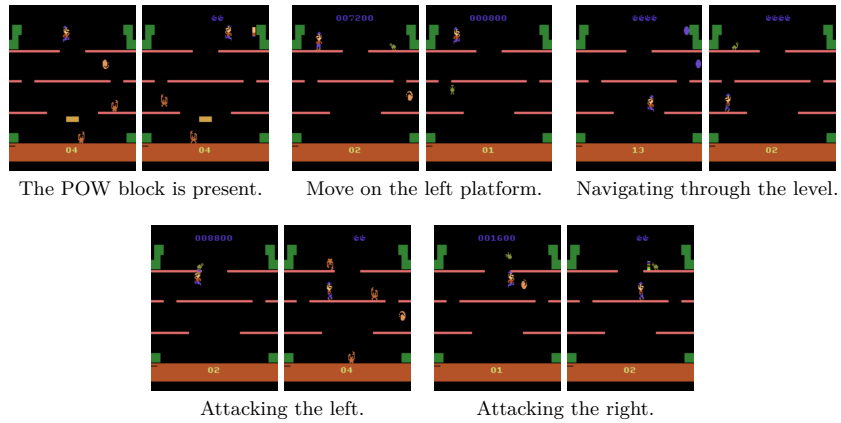


Figure 2.6: Effect of the number of clusters on the groupings identified by CAR in the Mario Bros environment.

Acronyms

643	APPO Asynchronous Proximal Policy Optimization 15
644	ARI Adjusted Rand Index 8–12, 23
645	CAR Clustering Agent Representation 1–5, 7–9, 11, 13–20, 23, 25
646	CS Concept Structure 1–3, 5
647	DL Deep Learning 14
648	DQN Deep Q-Networks 15
649	GPU Graphics Processing Unit 17
650	LR Latent Representation 1–3, 5, 9
651	LS Latent Space 1–5, 7–9, 11–13, 22, 23
652	MSE Mean Squared Error 4, 6
653	NN Neural Network 1–4
654	PPO Proximal Policy Optimization 15, 20–22
655	RI Rand Index 10
656	RL Reinforcement Learning 1, 4, 13–15, 17

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [AtariAge, 2024] AtariAge (2024). Mario bros. atari 2600. **Web**.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Hubert and Arabie, 1985] Hubert, L. and Arabie, P. (1985). Comparing partitions. Journal of classification, 2(1):193–218. **Article**.
- [Knuth and Moore, 1975] Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial intelligence, 6(4):293–326. **Article**.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.

- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS).
- [Peyré and Cuturi, 2020] Peyré, G. and Cuturi, M. (2020). Computational optimal transport. **Article**.
- [Rand, 1971] Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. Journal of the American Statistical association, 66(336):846–850.
- [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine learning. <https://github.com/rapidsai/cuml>.
- [Ross and Bagnell, 2010] Ross, S. and Bagnell, D. (2010). Efficient reductions for imitation learning. In Teh, Y. W. and Titterton, M., editors, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 661–668, Chia Laguna Resort, Sardinia, Italy. PMLR. **Article**.
- [Ross et al., 2011] Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In Gordon, G., Dunson, D., and Dudík, M., editors, Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, volume 15 of Proceedings of Machine Learning Research, pages 627–635, Fort Lauderdale, FL, USA. PMLR. **Article**.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.
- [Shlens, 2014] Shlens, J. (2014). Notes on kullback-leibler divergence and likelihood. **Article**.
- [Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press. **Book**.
- [Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread. **Web**.

- 732 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
733 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
734 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
735 environments.
- 736 [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying
737 the black box: Understanding dqns. **Article**.